G4DA COMPETITION · PAPER TRACK SUBMISSION · EXPERIMENTAL METHODOLOGY & AGENT TELEMETRY

qb-cubie in a Test-Guided Coding Agent: Methodology, Development, and Exploratory Pilot Findings

A Cryptographically Frozen A/B Comparison Between Example-Based Testing (qb-cubie 0.0.3) and pytest Across Single-Agent (v1–v5) and Multi-Agent (v6–v9) Architectures

Author & Project Lead: Dominick Wirzba · **Paper Synthesis Origin:** Gemini-App (Gemini-4-Argon) · **Post-Hoc Telemetry Analyses:** Antigravity (Gemini-3.8-Flash) & Codex (GPT-6.1-Sol)

Data Cutoff: November 12, 2026 (including v1–v5 baseline archive `20261111-200641` and multi-agent follow-up archives through `20261112-035522`) · **Open-Source & Reproducibility:** Published on PyPI (`qb-cubie`) and GitHub (MIT License) with complete frozen run archives and git history.

ABSTRACT — This study investigates whether access to the declarative quality assurance framework `qb-cubie`—including its *Example-Based Testing* (EBT) engine and structured guidance—helps a test-guided coding agent resolve real-world Python modification tasks under a fixed working budget compared to standard `pytest`. Both conditions operate with the same local model (`gemma4:12b`, 11.9B, GGUF `Q4_K_M` via Ollama 0.40.2), identical task snapshots, and identical tools, rules, and resource limits within each pilot. An independent evaluator assesses exclusively the filtered production patch in a fresh, Cubie-free container against pre-frozen private `FAIL_TO_PASS` and `PASS_TO_PASS` tests; visible tests modified by the agent cannot alter this benchmark.

Across the primary single-agent development series (generations v1 through v5; seven completed pilots comprising 42 agent runs on three benchmark tasks with Seed 42, backed by 5,563 verified SHA-256 hash references), privately confirmed resolutions were achieved in three runs (7.1%): twice with `pytest` (v2 and v4) and once in the `qb-cubie` condition (revised v3). In the v5 pilot with reasoning enabled (`think=true`), both conditions resolved 0 of 3 tasks: 11 of 67 model calls terminated at the output cap (`num_predict=8192`) emitting exclusively thinking tokens without an executable JSON action—consuming 87.3% (90,112 of 103,235) of all recorded output tokens and 79.6% of total agent runtime. No run in v5 successfully modified production code or executed an EBT contract (across all 21 Cubie runs in v1–v5, EBT was executed in only 2 runs). Independent post-hoc root-cause analyses (Antigravity and Codex) demonstrate that this 0:0 outcome reflects a severe operational **floor effect** caused by upstream inference and workflow bottlenecks rather than methodological equivalence. Finally, an exploratory follow-up series testing four serial multi-agent architectures (generations v6–v9: Analysis, QA, and Dev across 17 runs, 1,441 model calls, and 8.55M input tokens) confirms that while infrastructural errors (such as pagination cursor traps and context loss) were reduced to zero by v9, a 12B model without forced controller handoffs remains locked in repetitive orientation loops (82.6% duplicate actions in v9), underscoring that baseline agent capability must be lifted above the operational floor before downstream testing frameworks can exert a measurable statistical effect.

1. Research Question, Object of Study, and Contract Semantics

The primary research question formulated in the study protocol (**Protocol, 2026**; **Study Report, 2026**) asks:

Does access to `qb-cubie` and its specified guidance improve the independently verified resolution rate of a test-guided coding agent powered by Ollama and `gemma4:12b` under a fixed working budget compared to the exact same agent using standard `pytest`?

What is measured is the **combined offering of `qb-cubie` including its documentation guide** versus `pytest` including its corresponding guide (**Study Guides, 2026**). The study does not isolate the pure library effect from the instructional prompt effect, nor does it isolate individual EBT features. When an agent fails to invoke `qb-cubie` in the treatment arm, that non-utilization remains an empirical outcome of the evaluated agent package (Intention-to-Treat), though it limits direct inferences regarding the causal efficacy of executed EBT contracts (**Study Report, 2026**).

1.1 Declarative Contract Semantics of qb-cubie (Version 0.0.3)

The open-source package `qb-cubie` (evaluated at version `0.0.3`, commit `5e50d22f80db90c4e8ae6e3aeedab6acdb981230`) integrates declarative acceptance testing (*Example-Based Testing*, EBT), natural-language review delegation (*Review-Based Testing*, RBT), execution arenas (`TempFS`, `PyVenv`), unified static analysis orchestrators (`AstLinter`, `CstLinter`), and an experimental benchmark suite (`qb_cubie.benchmarks` / `cubie-bench`) (**Contract Semantics, 2026**; **Benchmark Guide, 2026**; **Linter Guide, 2026**; **RBT Usage Pattern, 2026**).

In EBT, a contract specifies a finite set of concrete executions of a callable `target` using mutable `WorldState` dataclasses with fields `args`, `kwargs`, `arena`, `fs`, `returns`, `expect`, `raises`, and `exception` (**Contract Semantics, 2026**). Let `P` denote the declared input states in `pre`, Q^+ the permitted target patterns (`post.expect=True`), and Q^- the forbidden target patterns (`post.expect=False`). Where `match(p, q)` indicates that the outcome observed when executing `p` satisfies all active comparisons in `q` (return values, expected exceptions via `ExceptionExpectation`, and recursive filesystem diffs in `fs`), the finite evaluation rule is defined as:

```
step_ok(p) = (Q+ is empty OR at least one q in Q+ matches)
             AND (no q in Q− matches)
run_ok     = all step_ok(p) hold
evaluation = run_ok if EBT.expect=True else NOT run_ok
```

While the arrangement “pre-state – operation – post-state” serves as a didactic analogy to a Hoare triple `{P} S {Q}`, `WorldState` is a concrete data record or pattern rather than a universally proven state predicate (**Quality Assurance Guide, 2026**). Passing an EBT contract establishes *observed conformance* over finite explicit inputs rather than a formal correctness proof or coverage metric. Because every input in `pre` is evaluated against the entire shared target set `post`, correlated input/output pairs such as `f(1)=2` and `f(2)=4` must be executed as separate EBT contracts (e.g., via `pytest.mark.parametrize`) so that swapped outputs are not falsely accepted (**EBT Guide, 2026**; **EBT Usage Pattern, 2026**). Furthermore, an omitted `returns` field or the sentinel `qb_cubie.UNSPECIFIED` disables return-value checking, whereas `WorldState(returns=None)` explicitly requires the return value `None` (**Contract Semantics, 2026**). To take effect in the study, an EBT contract must be executed via `.run()` and its result explicitly verified (via `assert result.evaluation is True` when `reporter=None`) (**Study Report, 2026**).

1.2 Independent Technical Audit and Circularity Prevention

To uphold strict scientific transparency, the framework was subjected to an internal critical code audit prior to submission (**Technical Audit, 2026**). While the repository test suite passes 2,569 tests (`2569 passed, 18 skipped, 6 warnings in 550.50s` on Linux, Python 3.13.12), the audit documented three technical defects in the core package: (B1, P1) `PyVenv` stores IPC files (`ipc_in.pkl`, `ipc_out.pkl`, `_cubie_phantom.py`) inside the workspace, which can overwrite identically named test fixtures (**Arena Management, 2026**); (B2, P1) `PytestReporter` relies on plain `assert` statements that are stripped under optimized execution (`python -O`); and (B3, P2) `CstLinter.check()` returns an empty list `[]` when passed a non-existent file path (**Technical Audit, 2026**). None of these three core findings affected the agent study runs.

Crucially, an early synthetic audit also highlighted the epistemological risk of circularity when validating a testing framework exclusively with its own contracts (**Technical Audit, 2026**). While the earliest test suite of `qb-cubie` originated via freehand bootstrapping combining informal RBT and EBT principles, all subsequent verification and regression suites—including the 698+ verification nodes developed for the study controllers—were deliberately written using standard, independent `pytest` assertions. Consequently, the existence of the repository's internal test suite is not claimed as empirical evidence that AI agents write better tests with `qb-cubie`.

2. Data Foundation and Scope of Investigation

The benchmark tasks originate from the local Gemma Developer Agent pool used with `swegemma` (**Data Documentation, 2026**). The source pool comprises 129 tasks across four Python repositories: 67 from FastAPI, 48 from Rich, 13 from Requests, and 1 from HTTPX. Because these tasks derive from public repositories, results do not represent an official SWE-bench Verified leaderboard submission (**Study Report, 2026**). A manual domain audit of 39 tasks documented in the candidate catalog (**Candidate Audits, 2026**) classified 26 tasks as qualified candidates, 9 as conditional, 3 as deferred, and 1 as excluded (with 90 tasks remaining semantically unaudited). From the audited pool, three tasks spanning distinct repositories and primary task types were frozen with Seed 42 for the pilot series (**Study Report, 2026**):

Task ID	Repository & Classification	Required Functional Modification	Private FAIL_TO_PASS / PASS_TO_PASS
`fastapi_9425`	FastAPI · API/Schema · Compatibility · In-process integration	Treat the stringified return annotation `"None"` in `get_typed_return_annotation` as a missing response model so that HTTP 204 bodiless responses succeed.	1 / 8
`requests_6589`	Requests · Parsing/Encoding · Bugfix · Local HTTP fixture	Account for UTF-8 encoded byte length rather than character count when computing string length in `super_len` (`Content-Length` path).	1 / 11

Task ID	Repository & Classification	Required Functional Modification	Private FAIL_TO_PASS / PASS_TO_PASS
`rich_3278`	Rich · Parsing/Encoding · Bugfix · Rendering snapshot	Strip private ANSI escape sequences in the range `ESC 0` through `ESC ?` so that they do not corrupt console output.	16 / 7

Prior to each run, repository snapshots are sanitized: git history and private reference files are stripped, symlinks are verified, and shared dependencies are pre-installed (**Study Report, 2026**). Agents cannot install packages or access an unrestricted shell; they receive only the public `problem_statement`. Section 4 and Section 5 evaluate all seven completed single-agent pilot archives under `runs/` (42 agent runs) plus one prepared-only archive (`20261111-180104`), while a separate 60-second technical smoke check (`local/live-pilot-validation`, capped at 3 calls) is tracked separately as an infrastructure test (**Study Report, 2026**). Section 7 then evaluates the five multi-agent follow-up archives (generations v6–v9, 17 executed runs) (**Multi-Agent Synthesis v6–v9, 2026**).

3. Research Methodology and Zero-Leakage Evaluation Harness

For each task and seed pair, both conditions (`cubie` and `pytest`) start from the exact same sanitized snapshot inside a fresh container with an empty dialogue history (**Study Report, 2026**). Task order is randomized deterministically by seed, and the starting condition alternates across tasks (in the frozen schedule for `20261111-200641`, Cubie starts on FastAPI and Rich, while pytest starts on Requests). Runs execute serially without sharing patches, dialogues, or evaluation signals. Both conditions share the exact same model, core instructions, structured JSON tools, context limits, and wall-clock budgets, differing solely in the pre-installed `qb-cubie 0.0.3` wheel and the framework guide (**Study Guides, 2026**).

3.1 Independent Evaluation Criterion

During an episode, both production code and visible tests remain freely editable by the agent (**Study Report, 2026**). Upon completion or timeout, the full patch and all test edits are archived, and a **fresh, shared evaluator container without Cubie** is launched. Using `swegemma 0.2.11`, the production filter strips all modifications to visible test files and protected runner configurations. Only the pure production patch is applied and tested against the pre-frozen private `FAIL_TO_PASS` (which fail with an assertion error on the base commit and pass on the reference fix) and `PASS_TO_PASS` regression tests (**Study Report, 2026**). Consequently, an episode is counted as resolved if and only if the isolated production patch passes all private target and regression tests—even if the agent stalled or timed out without calling `submit`. Conversely, weakening visible tests to obtain a green local run cannot pass private evaluation.

3.2 Frozen Runtime Configuration and Evidence Verification

Across all eight single-agent archives under `runs/`, **5,563 SHA-256 hash references** covering input manifests, preflight files in `ready.json`, and run manifests were audited; 100% matched without discrepancy (**Study Report, 2026**). The primary endpoint is the paired resolution rate difference (`Cubie minus pytest`), accompanied by an exploratory 95% bootstrap confidence interval resampling tasks with replacement (10,000 draws). The frozen environment for the v5 baseline pilot (`20261111-200641`) is summarized below:

Parameter	Frozen Specification in Archive `20261111-200641`
Model & Serving	`gemma4:12b` (11.9B, GGUF, `Q4_K_M`), served locally via Ollama `0.40.2`
Model Digest (SHA-256)	`4eb23ef187e2c5462566d6a1d3bbbc2f1346d0b4327cbb66d58fffbcc9b2b05c`
Sampling & Generation	Seed `42`, Temperature `0.2`, `think=true`, `num_ctx=32768`, `num_predict=8192`
Time & Call Budgets	`900 s` agent runtime per attempt; `90 s` single command; `120 s` private verification; unlimited call count
Context & Output Caps	Configured: `80,000 bytes` context / `12,000 chars` tool output; effective runner capacity: `40,090 bytes`
Agent Core & Guides	`testguided-core-v5` paired with `pytest-guide-v5` or `cubie-guide-v5`
Container & Bridge	Python `3.13.18`, `pytest 9.2.0`, Cubie image with `qb-cubie 0.0.3`; evaluator bridge via `swegemma 0.2.11`

4. Single-Agent Evolution and Results of the Core Pilot Series (v1–v5)

Following initial container replay validations (**Validation Report, 2026**), the single-agent architecture progressed through five frozen iterations (**Agent Design v1–v5, 2026**): from an early unversioned core (`600 s`, max 24 calls, `16,384 / 4,096` tokens), to **v2** (explicit working ledger, anchored replacements, stall recovery, and output repair), **v3** (action-coupled notes, evidence-based context, separate path/content search, baseline counter-checks, and EBT runtime tracking; expanded in revised v3 to `900 s`, unlimited calls, and `32,768 / 8,192` tokens), **v4** (runner-bound file hashes instead of model-supplied hashes, small-diff guidance, explicit oracle checks, and restricted read recovery), and **v5** (mandatory structured decision points for reproduction analysis, post-failure diagnosis, and pre-submission review, combined with enabling `think=true`).

4.1 Complete Overview of Single-Agent Pilot Archives (November 11, 2026)

Archive ID	Core / Thinking	Agent Time / Call Cap · Context / Output Tokens	pytest Resolved	Cubie Resolved	Complete / Unavailable Pairs
`132401`	early / off	600 s / 24 · 16,384 / 4,096	0 / 3	0 / 3	3 / 0
`160158`	early / off	600 s / 24 · 16,384 / 4,096	0 / 3	0 / 3	3 / 0
`163110`	v2 / off	600 s / 24 · 16,384 / 4,096	**1 / 3**	0 / 3	3 / 0
`171515`	v3 / off	600 s / 24 · 16,384 / 4,096	0 / 3	0 / 3	3 / 0
`180104`	v3 / off	prepared only (no agent runs executed)	—	—	—
`180702`	v3 revised / off	900 s / unlimited · 32,768 / 8,192	0 / 3	**1 / 3** (1 unavail.)	2 / 1
`192412`	v4 / off	900 s / unlimited · 32,768 / 8,192	**1 / 3**	0 / 3	3 / 0
`200641`	v5 / on (`think=true`)	900 s / unlimited · 32,768 / 8,192	0 / 3	0 / 3	3 / 0

4.2 Analysis of Verified Resolutions, Sensitivity Check, and Baseline Pollution

- **v2 Resolution (Archive `163110`, `requests_6589` with `pytest`):** The pytest agent resolved `requests_6589`, demonstrating a focused red–modification–green cycle and submitting cleanly (**Pilot Analyses, 2026**).
- **Revised v3 Resolution & Exit-Code-4 Sensitivity (Archive `180702`, `fastapi_9425` with `qb-cubie`):** The Cubie agent modified `get_typed_return_annotation` to handle `"None"`, authored and executed genuine EBT contracts (6 EBT executions and evaluations, 5 passed across test runs), and submitted a privately verified solution (**Study Report, 2026**). In the same archive, the Cubie agent on `requests_6589` (run `0004`) executed 3 EBT contracts but introduced an indentation error into production code, causing the private evaluator to fail during collection with pytest exit code 4. Historically, this run was recorded as `resolved=null` and `available=false`, yielding a paired effect of **+50.0 percentage points** (bootstrap CI [0; +100]) across two complete pairs. Counting this agent-induced syntax failure transparently as an unsuccessful resolution yields three complete pairs and a descriptive difference of **+33.3 percentage points** (**Study Report, 2026**).
- **v4 Resolution Without Process Integrity (Archive `192412`, `fastapi_9425` with `pytest`):** The production patch passed all 9 private tests (−33.3 percentage points, bootstrap CI [−100; 0]), but the agent overwrote a visible 1,114-line test file with 45 lines of broken tests, ending as `stalled` without submission. Only the independent production filter preserved the valid fix. Zero EBT contracts were executed across all three Cubie runs in v4 (**Pilot Analyses, 2026**).

- **Visible Baseline Pollution:** Preflight checks (`preflight/visible`) revealed substantial pre-existing failures in the full visible suites before any agent edits: FastAPI exhibited 8 collection errors (missing optional dependencies such as `pydantic_settings`); Requests had 577 passed, 5 failed, and 15 skipped tests; and Rich had 746 passed, **65 failed**, and 24 skipped tests (due in part to a snapshot inconsistency in `Column(highlight=...)`) (**Technical Analysis, 2026**).

5. In-Depth Analysis of Pilot v5 (Archive 20261111-200641)

In archive `20261111-200641` (core v5, `think=true`), all six runs completed private evaluation with 0/3 resolutions for `pytest` and 0/3 for `qb-cubie` (paired difference 0 percentage points; degenerate bootstrap interval [0; 0], which reflects zero variance across three 0:0 pairs rather than statistical equivalence) (**Study Report, 2026**):

Run	Task / Condition	Agent Time	Model / Tool Calls	Termination Reason	Final Patch Status
`0001`	FastAPI / Cubie	727.4 s	8 / 6	`invalid_output`	empty (no file modified)
`0002`	FastAPI / pytest	681.4 s	6 / 4	`invalid_output`	empty (no file modified)
`0003`	Requests / pytest	829.3 s	17 / 15	`invalid_output`	test additions only (`super_len("€")`)
`0004`	Requests / Cubie	900.1 s	19 / 17	`agent_timeout`	test edit only (surrogate literal)
`0005`	Rich / Cubie	653.5 s	8 / 6	`invalid_output`	empty (no file modified)
`0006`	Rich / pytest	684.7 s	9 / 7	`invalid_output`	empty (no file modified)

Across 4,476.4 seconds (74.6 minutes) of pure agent runtime (80.3 minutes total study wall time; median agent time 684.7 s for `pytest` vs. 727.4 s for `Cubie`), the two arms executed 67 model calls (32 `pytest`, 35 `Cubie`) and 55 tool calls (26 `pytest`, 29 `Cubie`), with 1 tool error in `pytest`, 4 tool errors in `Cubie`, and 0 EBT executions (**Study Report, 2026**). Complete token counts for `pytest` totaled 252,363 tokens (198,821 input / 53,542 output); due to a provider timeout on the final call of run `0004`, `Cubie` recorded a partial sum of 254,210 tokens (combined documented total: 506,573 tokens; 403,338 input and 103,235 output).

Reasoning Token Starvation: Out of 67 model calls, **11 responses terminated at the output cap (`done_reason=length` at 8,192 tokens) with an empty `message.content` and exclusively internal thinking output** (**Study Report, 2026**). These 11 truncated calls consumed 90,112 output tokens—**87.3% of all known output tokens** in v5 and 79.6% of total agent runtime (**Codex Analysis, 2026**). In FastAPI and Rich, both arms failed after reading source files before completing a single reproduction analysis or edit.

Micro-Mechanics on `requests_6589`: In run `0003` (`pytest`), the agent authored two accurate UTF-8 reproduction tests—`super_len("€") == 3` and `super_len("Test€") == 7`—which failed against the unpatched code with character counts `1` and `5`; however, its subsequent production edit failed with `edit_conflict/anchor_missing` before the final call hit the 8,192-token thinking cap (**Study Report, 2026**). In run `0004` (`Cubie`), the agent selected the 4-byte UTF-8 emoji 🚀, but after syntax and anchor rejections wrote the literal `'\ud83d\ude80'` (two UTF-16 surrogates, length 2) using a plain `assert` statement without any EBT contract, failing with `2 == 4` before timing out (**Study Report, 2026**).

6. Synthesis of Root-Cause Hypotheses (Antigravity & Codex) and Qualitative Context

Following the v1–v5 series, two independent AI systems conducted post-hoc root-cause analyses across all 42 runs: **Antigravity** (Gemini-3.8-Flash, Reasoning: High; **Antigravity Analysis, 2026**) and **Codex** (GPT-6.1-Sol, Reasoning: High; **Codex Analysis, 2026**). Both converge on the diagnosis that the 7.1% resolution rate (3/42) and the 0:0 outcome in v5 represent a structural **floor effect**: in 19 of 21 Cubie runs (90.5%), no EBT contract was ever executed because upstream inference, context, and interface barriers terminated the workflow before test methodology could take effect (**Study Report, 2026**).

6.1 The Five-Tier Bottleneck Cascade (Antigravity: $H_{Cascade}$ and H1–H4)

- **H1 (Token Starvation / Reasoning Displacement):** In v5's shared output budget (`num_predict=8192`), unbounded thinking displaced JSON action emission (87.3% output token loss; 11/67 responses or 16.4% of all calls in v5). *Falsifiable prediction:* Capping reasoning tokens (e.g., max 2,048) or raising `num_predict` to ≥ 16,384 tokens reduces length-truncated `invalid_output` failures below 10% (**Antigravity Analysis, 2026**).
- **H2 (Prior Dominance / Pre-Training Bias):** Trained overwhelmingly on standard `pytest` syntax, a 12B model under cognitive load defaults to plain `assert` statements whenever unrestricted file writing (`write_file`) is permitted, especially on stateful/fixture tasks (Requests, Rich) as opposed to pure functions (FastAPI in `180702`). *Falsifiable prediction:* Restricting test authoring to a dedicated contract-creation tool raises EBT execution from 9.5% (2/21) to over 75% (**Antigravity Analysis, 2026**).
- **H3 (Baseline Pollution / Noise Attractor):** Complete failure on `rich_3278` (0/14 across all pilots) is driven by the 65 pre-existing failures in the visible test suite acting as a cognitive distractor. *Falsifiable prediction:* Isolating `run_tests` to the task-specific test path prior to the first patch reduces stalling on Rich by at least 50% (**Antigravity Analysis, 2026**).
- **H4 (Capability Threshold / Model Boundary):** Simultaneously synthesizing domain logic, exact anchor edits, and valid test oracles exceeds the reliability threshold of 12B `Q4_K_M`. *Falsifiable prediction:* Evaluating a 30B+ or specialized coding model under the exact same freeze increases resolution on the three tasks from 7.1% to above 30% (**Antigravity Analysis, 2026**).

6.2 Workflow Bottleneck Hypothesis & Factorial Program (Codex) and Project Lead Assessment

Codex formulates a complementary **Workflow & Interface Bottleneck Hypothesis** and specifies a four-stage factorial verification program (**Codex Analysis, 2026**): **Study A** (a 2×2 design crossing `v4 vs. v5` × `think=false/true` to deconfound control prompts from reasoning), **Study B** (handle-bound local edit recovery to eliminate `anchor_missing`), **Study C** (model scaling under identical budgets), and **Study D** (clean vs. polluted visible baseline). Per the campaign qualification rules (**Study Report, 2026**), both arms must achieve private resolutions with a verified red–green cycle on at least two independent task groups before launching confirmatory hold-out measurements.

Project Lead's Perspective & Qualitative Observations: Separated from the quantitative metrics, the Project Lead notes that the current sample (three tasks, high floor effect) is insufficient for a definitive statistical verdict on `qb-cubie`, hypothesizing that a stronger model and/or refined orchestration is required (**Study Report, 2026**). In human-in-the-loop development on internal projects, however, `qb-cubie` has already proven highly valuable: EBT contracts are substantially easier for human developers to read, review, and audit for oracle correctness than procedural `pytest` code, and capable agent configurations can set up precise EBT tests rapidly. Future iterations will measure these readability and review-time endpoints explicitly (**Study Report, 2026**).

7. Concluding Follow-Up Study: Multi-Agent Generations v6 to v9

To test whether architectural role specialization and deterministic controller guardrails could lift `gemma4:12b` above the single-agent floor effect without changing the base model, four rapid follow-up generations (**v6, v7, v8, and v9**) were implemented, verified against up to 146 deterministic EBT/runtime test nodes (**Multi-Agent Designs v6–v9, 2026**), and evaluated on the same three task pairs (Seed 42, `think=false`, `temperature=0.15`, `num_ctx=32768`, `num_predict=8192`, up to 1,800 s wall-clock budget). Across all four generations, work was divided serially across three isolated role contexts sharing the workspace: **Analysis** (orientation, context curation, and final review), **QA** (test suite maintenance and EBT/pytest case authoring), and **Dev** (test execution and production code repair) (**Multi-Agent Synthesis v6–v9, 2026**).

7.1 Quantitative Overview of Multi-Agent Archives (v6–v9)

Generation & Archive ID	Run Status	Planned / Exec. Runs	Model / Tool Calls	Tokens (Input / Output)	Reached Roles (Calls)	Dominant Termination Cause	Resolved
v6 (`20261112-002038`)	`finished`	6 / 6	66 / 65	422,512 / 16,749	Analysis (100%)	6× `stagnation` (early controller stop after 3+3 idle steps)	0 / 6

Generation & Archive ID	Run Status	Planned / Exec. Runs	Model / Tool Calls	Tokens (Input / Output)	Reached Roles (Calls)	Dominant Termination Cause	Resolved
v7-1 (`20261112-013246`)	`failed`	6 / 3	46 / 45	316,839 / 5,649	Analysis (18), QA (27)	`ValueError` (post-freeze code modification guard)	0 / 3
v7-2 (`20261112-015457`)	`interrupted`	6 / 1	81 / 80	638,637 / 5,705	Analysis (100%)	Manual abort (36× duplicate `find_paths` loop after tool removal)	0 / 1
v8 (`20261112-030527`)	`interrupted`	6 / 1	64 / 63	195,452 / 2,302	Analysis (61), QA (2)	Manual abort (30+ `cursor_error` loop during `ask_analysis`)	0 / 1
v9 (`20261112-035522`)	`finished`	6 / 6	1,184 / 1,179	6,980,998 / 44,845	Analysis (100%)	6× `budget_exhausted` (1,800 s wall-clock limit; 974 duplicates)	0 / 6
Total (v6–v9)	—	**30 / 17**	**1,441 / 1,432**	**8,554,438 / 75,250**	**Analysis dominant**	—	**0 / 17**

7.2 Architectural Progression and Emergent System Dynamics Across v6–v9

Across all 1,432 executed tool actions in v6–v9, 90.7% consisted solely of `search_content` (795 calls, 55.5%) and `read_file` (504 calls, 35.2%), alongside 72 `read_evidence` calls (5.0%) and 52 `find_paths` calls (3.6%), while only 3 `create_case` and 1 `collect_tests` calls occurred (all in v7-1) (**Multi-Agent Synthesis v6–v9, 2026**). Tracking the four iterations reveals a clear **Error Domain Migration** and a fundamental **Handoff Paradox** for compact 12B models:

1. **v6 (`20261112-002038`) — Two-Step Handoff Barrier & Silent Scope Misses:** Disabling reasoning (`think=false`) immediately restored JSON action stability (65 of 66 calls returned compact valid JSON; a single 8,192-token regex escape loop in `rich_3278` was repaired via `output_repairs`) (**v6 Pilot Analyses, 2026**). However, all 6 runs terminated early via `stagnation` (45 s to 531 s) inside `analysis/orient` without a single call to QA or Dev. On `requests_6589`, searching in `requests/utils.py` instead of `src/requests/utils.py` returned silent empty lists (`hits: []`), and the model never initiated the two-step protocol, confirming Counter-Hypothesis V6-H8 (handoff overhead blocking progress) ().

2. `publish_context → handoff` **v6 Pilot Analyses, 2026**
3. **v7 (`20261112-013246` & `20261112-015457`) — Context Pollution & Schema Sabotage:** v7 introduced a pre-computed repository start card, explicit `scope_not_found` candidate suggestions, atomic `finish_orientation`, and removed early stagnation stops (**Multi-Agent Designs v6–v9, 2026**). While v7-1 briefly reached QA (defining 3 test cases before aborting on a post-freeze check), v7-2 exposed severe context contamination: passing raw JSON telemetry ledgers (with SHA-256 hashes and float metrics) alongside 45 lines of Cubie tutorials inside the Analysis prompt caused the model to lose track of 55 truncated tool exchanges. When recovery temporarily removed `search_content` from the schema, the model misused the filename glob tool `find_paths` to search for code (`"pattern": "def get_route_info"`) 36 times in a row (**Multi-Agent Synthesis v6–v9, 2026**).
4. **v8 (`20261112-030527`) — Forced Controller Handoff vs. Epistemic Amnesia & Cursor Traps:** v8 stripped the Cubie tutorial from Analysis, introduced a compact single-step `finish` action, and implemented a deterministic forced handoff (`repeat_count >= 3 → activate QA`) (**Multi-Agent Designs v6–v9, 2026**). This forced handoff successfully activated QA, which issued two targeted `ask_analysis` queries (`job-12`, `job-20`). In `job-12`, Analysis located the exact target functions in `fastapi/dependencies/utils.py` (lines 253–280, `file-15`, `file-17`). However, because the model did not manually invoke `update_context` before finishing, the discovered files were lost (`materials: []` in `job-20`). Simultaneously, omitting the explanation of the pagination parameter `cursor` caused the model to pass `"cursor": "0"` or file paths, triggering over 30 consecutive `cursor_error` rejections (**Multi-Agent Synthesis v6–v9, 2026**).
5. **v9 (`20261112-035522`) — Infrastructural Perfection vs. Autonomous Orientation Lock-In:** v9 eliminated `update_context` in favor of transactional Zero-Admin Memory (automatically persisting all reads, searches, and notes), removed `cursor` from all model schemas, granted all three roles symmetric read/search autonomy, rendered prompts as clean four-section Markdown, and removed the v8 forced handoff so that Analysis would transition autonomously via `finish` (**Multi-Agent Designs v6–v9, 2026**). Over 3 hours and 6 minutes of execution (1,184 model calls, 1,179 tool calls, 6.98M input tokens), infrastructural errors dropped to **0 cursor errors, 0 tool errors, and 0 context overflows** (**v9 Pilot Analysis, 2026**). Moreover, Analysis located the exact target files within steps 0–4 on Requests, steps 4–9 on FastAPI, and steps 0–13 on Rich. Yet across all 1,184 responses, the model invoked `finish` **zero times**: because `finish` was overloaded in the prompt with review parameters (`verdict=verified/rework`) and no forced controller transition was active, all six runs remained in `analysis/orient` for the full 1,800 seconds, generating 974 duplicate actions (82.6%, including reading the same lines of `tests/test_utils.py` 62 times in run `0003` and searching `strip_control_codes` 67 times in run `0005`) (**v9 Pilot Analysis, 2026**). Because the framework guide in v9 is injected only upon activating QA (`team_v9.py:615`), both arms remained methodologically invariant at 0/3 resolutions (**v9 Pilot Analysis, 2026**).

8. Overall Conclusion

Across nine iterative agent generations (59 executed runs: 42 in single-agent pilots v1–v5 and 17 in multi-agent pilots v6–v9), this study provides both a rigorously audited, zero-leakage benchmarking methodology for evaluating developer tools like `qb-cubie` and a comprehensive empirical mapping of the failure modes of compact 12B open-weights models under test-guided constraints (**Study Report, 2026**; **Multi-Agent Synthesis v6–v9, 2026**). While flat single-agent architectures (v2–v4) achieved the only three privately verified repository repairs—including a complete proof-of-concept resolution executing six EBT contracts in archive `180702`—both unbounded reasoning in v5 (87.3% output token starvation) and autonomous multi-agent handoff protocols in v6–v9 (82.6% duplicate orientation loops in v9 despite rapid code localization and zero tool errors) demonstrate that 12B models fail primarily at metacognitive phase transitions and interface boundaries rather than initial code retrieval. Lifting agent performance above this operational floor via bounded controller state transitions, dedicated contract-generation tools, and higher-capacity models forms the direct empirical roadmap for subsequent confirmatory evaluations of `qb-cubie`.

9. References and Frozen Study Artifacts

1. **Study Report (2026):** Wirzba, D. *qb-cubie in einem testgeführten Programmieragenten: Methodik, Entwicklung und explorative Pilotbefunde* (`experiments/testguided-study/PAPER.md`, data cutoff: November 11, 2026, including archive `runs/20261111-200641`).
2. **Protocol (2026):** *Studienprotokoll der gepaarten testgeführten Agentenstudie* (`experiments/testguided-study/PROTOKOLL.md`).
3. **Data Documentation (2026):** *Herkunft und Struktur des Aufgabenpools* (`experiments/testguided-study/DATEN.md`).
4. **Candidate Audits (2026):** *Fachliche Einzelprüfungen und Kandidatenkatalog* (`experiments/testguided-study/KANDIDATEN.md` and `candidate-catalog.json`).
5. **Study Guides (2026):** *Eingefrorene Agentenanleitungen für qb-cubie und pytest* (`experiments/testguided-study/guides/cubie.md` and `guides/pytest.md`).
6. **Validation Report (2026):** *Technische Durchstich- und Replay-Validierung der Studienbrücke* (`experiments/testguided-study/VALIDIERUNG.md`).
7. **Agent Design v1–v5 (2026):** *Entwicklungshistorie der Agentenkerne v1 bis v5* (`AGENTENDESIGN.md`, `AGENTENDESIGN-V4.md`, and `AGENTENDESIGN-V5.md`).
8. **Pilot Analyses (2026):** *Detaillierte Auswertungen der Pilotläufe v2 bis v4* (`PILOTANALYSE-20261111.md` and `PILOTANALYSE-20261111-192412.md`).
9. **Technical Analysis (2026):** *Analyse der sichtbaren Ausgangstests und Snapshot-Vorbelastungen* (`TECHNIKANALYSE-20261111-171515.md`).
10. **Antigravity Analysis (2026):** *Post-hoc-Ursachenanalyse und Kaskaden-Hypothesensystem (Gemini-3.8-Flash, Reasoning: High)* (`HYPOTHESEN-URSACHENANALYSE-20261111.md`).
11. **Codex Analysis (2026):** *Post-hoc-Ursachenanalyse und Workflow-Prüfprogramm (GPT-6.1-Sol, Reasoning: High)* (`URSACHENANALYSE-CODEX-20261111.md`).
12. **Benchmark Guide (2026):** *Benchmarks: Einstieg, eigene Agenten und Messbetrieb* (`docs/GUIDE_Benchmarks.md`, package `qb_cubie.benchmarks` / CLI `cubie-bench`, November 11, 2026).

13. **Contract Semantics (2026):** *Verbindliche Vertragssemantik von Cubie 0.0.3* (`docs/GUIDE_Contract-Semantics.md` , aligned with `src/qb_cubie/ebt.py` , `protocols.py` , and `rbt.py`).
14. **Quality Assurance Guide (2026):** *Qualitätssicherung mit Cubie: Beobachtung, Abdeckung und Beweis* (`docs/GUIDE_Quality-Assurance.md`).
15. **EBT Guide (2026):** *Example-Based Testing: Ausgangsdaten, Zielmuster und Unterspezifikation* (`docs/GUIDE_Example-Based-Testing.md`).
16. **EBT Usage Pattern (2026):** *EBT-Nutzungsmuster und ausführbare Vertragsbeispiele* (`docs/GUIDE_EBT-Usage-Pattern.md`).
17. **Arena Management (2026):** *Arena-Management: TempFS, PyVenv, Hydrierungsgrenzen und Laufzeitlimits* (`docs/GUIDE_Arena-Management.md`).
18. **RBT Usage Pattern (2026):** *RBT-Nutzungsmuster: Interaktive und automatisierte Reviewer* (`docs/GUIDE_RBT-Usage-Pattern.md`).
19. **Linter Guide (2026):** *Linter: AstLinter, Pylint/MyPy-Verträge und CstLinter-Regeln* (`docs/GUIDE_Linter.md`).
20. **Technical Audit (2026):** *Kritisches Gutachten zu qb-cubie* (`GUTACHTEN.md` , November 11, 2026, commit `5e50d22f80db90c4e8ae6e3aeedab6acdb981230` , version `0.0.3`).
21. **Multi-Agent Designs v6–v9 (2026):** *Konzepte und Implementierungen der Generationen v6 bis v9* (`AGENTENDESIGN-V6.md` , `AGENTENDESIGN-V7.md` , `IMPLEMENTIERUNG-V7.md` , `AGENTENDESIGN-V8.md` , `AGENTENDESIGN-V9.md` , and `IMPLEMENTIERUNG-V9.md`).
22. **v6 Pilot Analyses (2026):** *Deskriptive Analysen des v6-Piloten 20261112-002038* (`PILOTANALYSE-V6-20261112-002038-CODEX.md` and `PILOTANALYSE-20261112-002038.md`).
23. **v9 Pilot Analysis (2026):** *Deskriptive Analyse der Agenten-Generation v9, Laufarchiv 20261112-035522* (`PILOTANALYSE-V9-20261112-035522.md` , Antigravity / Gemini-3.8-Flash).
24. **Multi-Agent Synthesis v6–v9 (2026):** *Deskriptive Analyse und vergleichende Gesamtschau der Agenten-Generationen v6, v7, v8 und v9* (`ANALYSE-GENERATIONEN-V6-V9.md` , Antigravity / Gemini-3.8-Flash, November 12, 2026).